

The cut-off mirrored double cone

Stage 1 as an hourglass — and a cone the model shapes itself

E-080 • design proposal • 15 September 2026 • **nothing here is measured yet**

The deck turns Chris's four points into a specification: a waist instead of a tip, a mirrored cone of targets, a random mix of four tasks, and a cone whose shape the model learns per region — one direction per flow, one scale per channel.



The cut-off mirrored double cone

This deck proposes the next shape of the stage-one model, the encoder that turns one place at one time into an embedding. Chris set four points this morning: the input cone loses its tip and becomes a bottleneck of several points; a second cone is sampled into the future and used only as targets; the model trains under a random mix of forecasting and reconstruction tasks; and the shape of the cone — where it points, how wide it is, which way it is stretched — becomes something the model learns separately for every region of the planet. The first three points are settled decisions and the deck turns them into a specification. The fourth is a request for a proposal, and the deck's recommendation is a warped sunflower: a fixed pattern of sample points that a small set of per-region numbers stretches and shifts, with one direction shared by everything a flow carries and one scale per channel, read through a soft aperture so that the loss can push on the numbers directly. Everything is written so it can be measured as one experiment, E-080, against the geometry we already have. Nothing in this deck has run.

One cone, five seeds, one clear reading

E-069 • the cone-native codec

- 30 days of dots around a North Atlantic cell, one 3×3 patch at the present, targets at +1 and +2 pentads
- History did **not** put the cell's own velocity into the embedding — five seeds, two objectives, refuted
- History **did** buy persistent fields (hidden sea-surface height at ~0.36 of the bar vs 0.75–0.90) and a 7–8 % better one- and two-pentad forecast

E-071 • cone v2

- Global, harmonic climatology, profile tokens — the geometry's known faults, fixed
- Reach from the fastest mechanism $\times 1.5$ — a hemisphere by lag 3, so dots go log-radial

The open question v2 left: a cone wide enough to never exclude a driver samples the near field at one dot per thousands of kilometres.

One cone, five seeds, one clear reading

A quick reminder of the record, because the new design is a response to it. E-069 was the first cone-native codec: for every ocean cell it read a thirty-day cone of samples around it, called dots, plus the present three-by-three patch, and it was asked to predict the cell's own future one and two pentads out — a pentad is a five-day bin. Its headline hypothesis, that thirty days of history would put the cell's own current velocity into the embedding, was refuted at five seeds under two objectives. What the history did buy was real: fields that persist, like sea-surface height, were reconstructed far better from history, and the short forecast improved by seven to eight percent. Then cone v2 fixed the geometry's known faults — it went global, it smoothed the seasonal baseline, it placed the Argo float tokens where floats actually were, and it sized every cone from the fastest thing that could carry information, times a safety factor. That last choice has a cost the v2 plan named itself: with a reach of a hemisphere by lag three, a fixed budget of dots samples the near field very thinly. The design in this deck is what lets the cone be both — wide enough to never exclude a cause, and dense where the causes actually are — by letting each region decide where its dense part points.

Past cone → waist → future cone



- **Past cone** (lags $-6 \dots -1$): read as input, dots at the per-lag ellipse plus the anchor column
- **Waist** (lag 0): a disc of several cells — the cut-off tip — read as input and reconstructed; the embedding z summarises the waist
- **Future cone** (leads $+1 \dots +6$): never read; sampled the same way and used as targets only
- **Point-mirrored**: the future cone at lead $+\ell$ is the past cone at lag $-\ell$ reflected through the anchor

Past cone → waist → future cone

This is the whole shape on one page. Time runs upward, space sideways. Below the middle is the past cone: everything the model reads about the last thirty days, widening with lag because a driver further back in time can have come from further away. In the middle is the waist — the tip of the cone, cut off. In E-069 the present was a single three-by-three patch; here it is a small disc of several cells, and it is the only place the model both reads and is asked to reconstruct. The embedding is made here, at the waist. Above the middle is the future cone: the same shape reflected through the anchor point, widening into the future, and never read — it exists only to be predicted. Mirrored means point-mirrored: the future cone at lead plus three is the past cone at lag minus three turned through the anchor. That is not a decoration; it is what advection looks like in space-time. Water that reaches the anchor came from upstream and will go downstream, so a cone that leans upstream in the past should lean downstream in the future. The dashed line is the anchor column — the cell's own history and its own future — and it is always inside both cones whatever else the geometry does, so the fixed-frame forecast that stage two needs is always a target.

A gradient does not exist within a single cell

- The waist is a disc of radius r_0 around the anchor, sampled as a small sunflower: ~13 cells at the default (one centre + 12), never fewer than the 3×3 patch's nine
- Read as input at lag 0 (values + observed flags) and reconstructed as the anchor family's target
- **What the waist buys:** thermal wind is a density gradient; geostrophy is a sea-surface-height slope — both need neighbours at the same instant, not in the past
- The waist has the same shape parameters as the cone at lag 0 (orientation, two semi-axes), so a boundary current can have an elongated waist along the flow
- The embedding z summarises the waist; one z per anchor, as today

≈ 13

cells at the waist

one centre + 12, never fewer than the nine of the 3×3 patch

1 z

per anchor, as today

so stage 2 and every archived probe see the same kind of object

A gradient does not exist within a single cell

Point one. Why cut the tip off at all? Because the quantities the AMOC read-out actually depends on are gradients. Thermal wind — the vertical shear of the current — is a horizontal density gradient. Geostrophic velocity is a slope in sea-surface height. A gradient does not exist inside one cell; it needs neighbours at the same moment. The E-069 record says the same thing from the other side: the snapshot twin's velocity read-out was exactly what a ridge regression on the raw three-by-three patch gives — geostrophy from the patch — and the cone added nothing to it. So the present deserves more than a single point. The waist is a small disc of cells around the anchor, thirteen by default, arranged as a tiny sunflower so that every bearing is represented. It is read as input, and it is what the model reconstructs in the anchor-reconstruction task. It carries the same shape parameters as the rest of the cone evaluated at lag zero, so where the flow is a narrow jet the waist can be stretched along it. One embedding per anchor summarises the waist, exactly as today, so stage two and every archived probe see the same kind of object.

Sampled like the past cone, read by nobody

+1 ... +6 pentads

30 days ahead, matching the 30 days of history

targets only

nothing in the future cone is ever read by the encoder

frozen at evaluation

the fixed v2 hourglass scores every arm and every seed

- Leads +1...+6 pentads (30 days ahead), the mirror image of the past cone through the anchor: centre $c(+\ell) = -c(-\ell)$, same axes and orientation at the same $|\ell|$
- Dots are **targets only**: each is a decoder query (channel, Δx , Δy , lead, depth) $\rightarrow (\mu, \log \sigma^2)$, scored with the Gaussian negative log-likelihood already in `cone_codec.py`
- The anchor column at every lead is always among the targets — the fixed-frame forecast stage 2 needs
- Per-lead and per-family loss, each reported against its own predict-the-mean bar (the E-069b rule)
- Evaluation geometry is **frozen** (the fixed v2 hourglass), whatever the training cone learns — so every number stays comparable across arms and seeds

Sampled like the past cone, read by nobody

Point two. The prediction cone is the future half of the hourglass. It is sampled by the same machinery as the past cone, reflected through the anchor, out to six pentads ahead — thirty days, matching the thirty days of history. Nothing in it is ever read by the encoder. Each dot in it is a question put to the decoder: at this channel, this offset, this lead, this depth, what is the value? The decoder answers with a mean and a variance, and is scored with the Gaussian negative log-likelihood we already use. Two rules carry over from E-069b, the run that fixed the first masking plan: every loss family is reported against the score you would get by predicting the mean, so a target that cannot be known shows up as a bar of one rather than hiding in an average; and the anchor's own column is always among the targets, so the forecast at the anchor itself — the thing stage two rolls forward — is always trained. One more rule is new and matters for point four: the geometry used at evaluation time is frozen to the fixed hourglass. The training cone may move; the yardstick does not. Otherwise two runs with different learned cones would be scoring themselves on different questions.

One model, four questions, drawn at random per example

task	input	targets	share	what it teaches
T1 forecast	waist + past cone	future cone	0.35	the full problem
T2 snapshot forecast	waist only (past cone fully hidden)	future cone	0.15	the built-in twin: what history buys, inside one model
T3 nowcast from history	past cone under a dropout pattern; waist hidden	waist	0.25	the present from the past — persistence, advection, memory
T4 fill-in	waist + past cone, both under a dropout pattern	the hidden waist cells and dots	0.25	E-069b's reconstruction family, kept

Dropout patterns for T3 and T4, drawn independently (each fires with its own probability)

channel drop

at the waist — lag 0 only,
chan_drop_scope lag0

recent-lag band

hide every dot at $\text{lag} \leq \ell_o, \ell_o \in \{1, 2, 3\}$

bearing wedge

hide a 90° sector

random dots

each dot hidden with $p = 0.3$

whole waist

T3 only

One model, four questions, drawn at random per example

Point three. Each training example draws one of four tasks. Task one is the full problem: read the waist and the past, predict the future cone. Task two is the same question with the past entirely hidden — the snapshot twin from E-069, but now living inside the same model, so every run reports what history buys without a second run. Task three is the nowcast: hide the present and reconstruct it from history, under one of several dropout patterns applied to the past. Task four keeps E-069b's reconstruction family: hide some cells and dots in both the waist and the past and fill them in. The dropout patterns are the ones already in the code plus one: dropping a channel at the present only, hiding the most recent lags so the model must extrapolate from stale history, hiding a wedge of bearings so it cannot lean on one direction, hiding dots at random to imitate sparse observations, and — for the nowcast only — hiding the whole present. The shares are a starting point, not a result; they are a knob the experiment records. The one rule that is not a knob comes from E-069b's failure: a target's channel must be visible somewhere in the input, at some lag, or the target is unknowable and only teaches the model to predict the mean.

Three rules, all learned the hard way in E-069

1 • Knowable targets only

A target channel must be visible somewhere in the input (any lag, any dot). Under the first E-069 plan 80 % of hidden-dot targets broke this and 64 % of the loss was predict-the-mean.

2 • No copy targets

The anchor family scores hidden channels only (anchor_hidden_only); a target visible in the input is a copy, not a question.

3 • Every family against its own bar

Per (task, family, lead) loss, each divided by its predict-the-mean score; a task is “learned” only where the ratio is below one at held-out anchors.

- The task is not told to the model by a token; the masks are visible (the mask_tok / miss_tok distinction stays), and the lead is in every query's coordinates — that is enough
- Loss weights: equal across tasks per example; the shares in the table set the mix, not the weights

Three rules, all learned the hard way in E-069

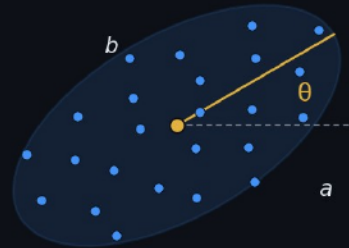
Three rules, all paid for already. The first is that every target must be knowable from the input: the first E-069 plan hid a channel at the present and at every dot, so most of the hidden-dot targets belonged to channels the encoder could not see at all, and two thirds of the loss became a lesson in predicting the mean. The second is that no target may be a copy of something visible: the anchor family scores only channels that were dropped, because a target the model can read off the input is not a question. The third is that every loss family is reported against its own predict-the-mean bar, per task, per family, per lead, so a number is only called learned where the ratio sits below one at held-out anchors. On mechanics: the model is not told which task it is in by a special token. The masks are visible — there is already a token for "hidden by us" and a different one for "the data never observed this" — and every decoder query carries its lead in its coordinates. That is enough for the model to know what is being asked. The shares in the table decide how often each task is drawn; within an example the loss weights are equal.

Eight numbers per region shape the whole hourglass — and none of them is in kilometres

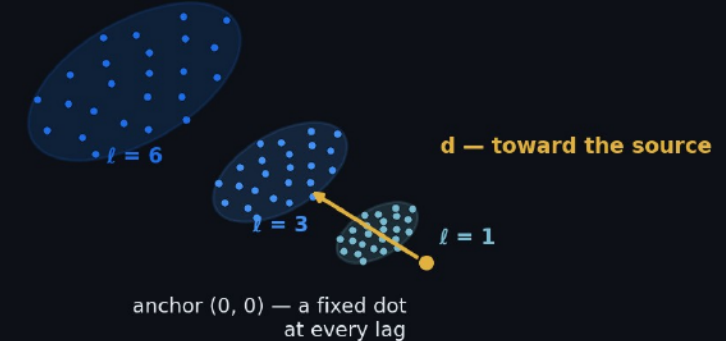
(a) canonical sunflower, unit disc
24 points, golden angle 137.508°



(b) rotated by $\theta = 30^\circ$, stretched to 2:1
semi-axes a, b



(c) the same pattern at three lags: shifted by $\ell \cdot d$, grown with ℓ



$$S = \log \Sigma = m \cdot I + k \cdot [\cos 2\theta, \sin 2\theta; \sin 2\theta, -\cos 2\theta]$$

For each region and channel group, the past cone at lag ℓ is a Gaussian footprint centred at anchor + $\ell \cdot d$ with covariance $\Sigma(\ell)$, and the dots are the canonical sunflower mapped through it:

$$\Sigma(\ell) = \Sigma_0 + \ell^2 \Sigma_v \quad p(\ell, i) = \text{anchor} + \ell \cdot d + \Sigma(\ell)^{1/2} \cdot u_i$$

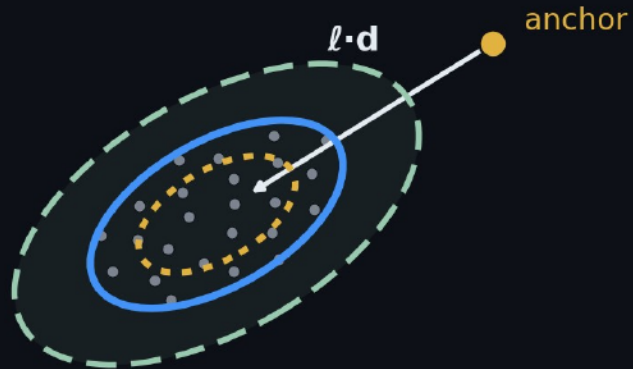
- u_i : the canonical sunflower on the unit disc (Vogel spiral, golden angle — the E-026 pattern in [ml/temporal.py::spiral_offsets](#)), fixed, 24 points, the same at every lag
- d (2 numbers): the drift per pentad — where the footprint's centre moves for each pentad back in time; **the arrow toward the source**. Stored dimensionless: $d = \tanh(x) \cdot v_{\text{design}} \cdot \Delta t$
- Σ_0 (3): the waist ellipse, stored as its **matrix logarithm** $S_0 = \log \Sigma_0$ — a symmetric 2x2 whose entries are in log-km. Its trace is the size; its traceless part is $(\log \text{aspect ratio}) \times (\cos 2\theta, \sin 2\theta)$. No angle, no 180° wrap; a circle is $S = m \cdot I$
- Σ_v (3): the velocity-spread ellipse, also as $\log \Sigma_v$ — how fast the footprint widens with lag (standard deviation $\propto \ell$, as the reach $\propto (1 + \ell)$ in [ml/cone.py::reach_km](#))
- **Why log form**: under Adam every parameter moves about one learning rate per step in its own units. A radius in km moves a fixed number of km (frozen at 850 km, twitchy at 30 km); a log-radius moves a fixed **fraction** — the same behaviour at any size
- **Future cone**: reflected — centre $-\ell \cdot d$, the same $\Sigma(\ell)$; the anchor column (0, 0) is always added. The per-channel scale is the next slide

Eight numbers per region shape the whole hourglass — and none of them is in kilometres

Point four, the proposal, in its revision-3 form. The cone at a given lag is a Gaussian footprint: a centre, and a covariance that says how wide it is and in which direction it is stretched. The dots are the fixed sunflower — the same twenty-four-point pattern the project has used since E-026, the experiment that introduced the sunflower stencil for stage two — pushed through that covariance, so the pattern is squashed, stretched and shifted but never re-invented. Eight numbers per region and channel group define it. Two are the drift: where the centre moves for each pentad you step back, the arrow toward the source, stored as a fraction of the design speed so it can never exceed it. Three are the waist ellipse, and here the storage matters: instead of an angle and two radii, we store the matrix logarithm of the covariance — a symmetric two-by-two matrix whose entries are in log-kilometres. Its trace is the overall size; its traceless part is the log of the aspect ratio times the double-angle phasor, $\cos 2\theta$ and $\sin 2\theta$. That representation has no angle to wrap around at 180 degrees, treats a circle as the natural zero, and — the point the reviewer raised — is scale-free. Three more numbers are the velocity-spread ellipse, which says how fast the footprint widens per pentad; a footprint whose standard deviation grows linearly with lag is exactly the reach-grows-with-lag rule the geometry module already encodes. Why log form, in one sentence: the Adam optimiser moves every parameter by roughly one learning rate per step in that parameter's own units, so a radius stored in kilometres moves a fixed number of kilometres — which freezes a wide aperture and makes a narrow one jitter — while a log-radius moves by a fixed fraction of its current size, the same at every scale. The future cone is the mirror image with the same covariance, the anchor's own cell is always included, and the one thing this slide leaves out — that channels need different sizes — is the next slide.

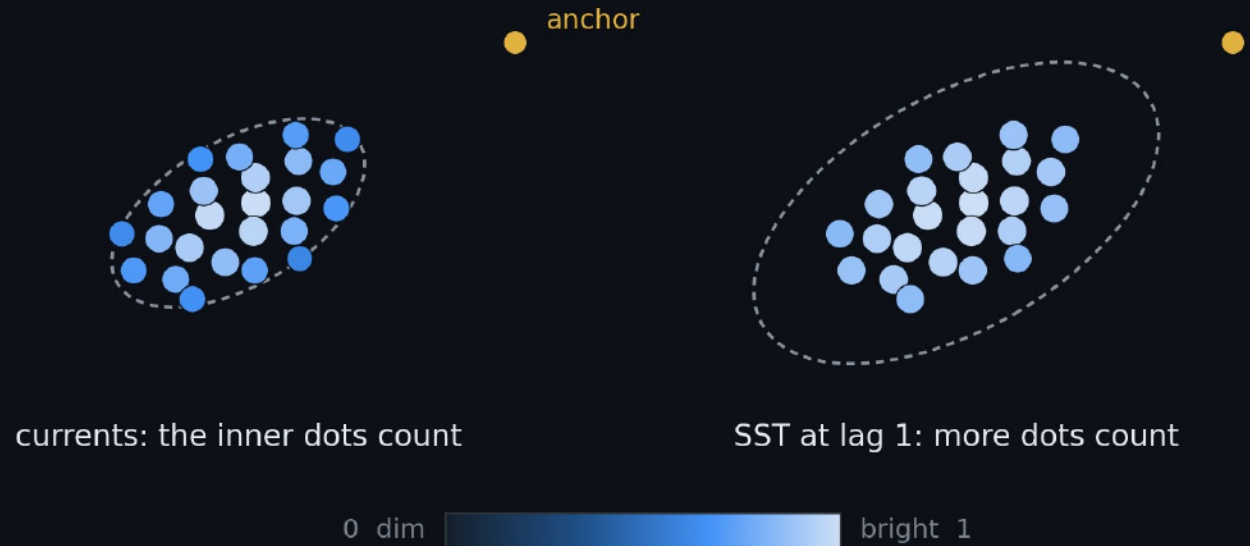
The shape and the drift belong to the flow; only the size belongs to the channel

(a) one dot set at lag 3, three apertures



- currents, sea-surface height · $s = 0$
- - sea-surface temperature at lags 0–1 · $s > 0$
(stirred by the atmosphere)
- - - a channel with $s < 0$

(b) the same dots, coloured by $w(c, i) = \exp(-\frac{1}{2} p^T \Sigma_c(\ell)^{-1} p)$



- **1 • Shared** per region, lag and channel group: the shape $\Sigma(\ell)$ and the drift d of slide 8. Everything the water carries is carried the same way — the arrow toward the source and the elongation along the jet are properties of the **flow**, not of the tracer. *One direction per flow.*
- **2 • Per channel:** one log-scale offset s_c (two for the L-shaped channels: lags 0–1 and 2–6). $S_c(\ell) = S(\ell) + s_c \cdot I$, i.e. $\Sigma_c(\ell) = \exp(s_c) \cdot \Sigma(\ell)$ — the same ellipse inflated or shrunk, **never turned**. *One scale per channel.*
- **3 • Initialised from the reach table** in [ml/cone.py::FAMILIES](#): wind stress 500 km at lags 0–1 and nothing beyond; currents and sea-surface height 129.6 km · (1 + ℓ); sea-surface temperature and mixed-layer depth max(that, 500 km) at lags ≤ 1 , then the ocean's growth; land 400 km flat. At step 0 every channel reads the reach the current geometry gives it

The shape and the drift belong to the flow; only the size belongs to the channel

This slide answers a question Chris asked: don't we need a scale for each channel? Yes — and the split is the whole design. What is shared is the direction: the drift toward the source and the elongation of the footprint along the jet. Those are properties of the flow. Temperature, salinity, sea-surface height and the currents themselves are all carried by the same water, so giving each of them its own arrow would multiply the parameters and let the model learn different "upstreams" for quantities that arrive from the same place. What is per channel is the scale. The current geometry module — the file ``ml/cone.py``, whose ``FAMILIES`` table assigns every channel a propagation speed, a memory and a correlation length — already says the channels differ: wind stress reads five hundred kilometres at the two lags inside its ten-day memory and nothing beyond, the ocean channels grow at a hundred and thirty kilometres per pentad, sea-surface temperature and mixed-layer depth are L-shaped — wide at lags zero and one where the atmosphere stirs them, then the ocean's slow growth — and land is a flat four hundred kilometres because soil does not advect. So each channel gets one number, a log-scale offset, that inflates or shrinks the shared ellipse without turning it; the L-shaped channels get two, one for the first two lags and one beyond. Those offsets start at the values the geometry module already uses, so at step zero every channel reads exactly the reach it reads today. In the picture: the left panel is one dot set with three apertures of different size but the same centre and orientation; the right panel shows the same dots weighted for two channels — for the currents only the inner dots count, for sea-surface temperature at lag one more of them do. How the weighting happens, and why it costs no tokens, is the next slide.

One token per dot carries every channel; each channel's entries are weighted by its own aperture before the projection

(c) token anatomy

one per-location token at dot i



- A **per-location token** (E-071 §6.4) carries all channels of a group at one dot position p_i : a vector of values and a vector of observed flags, one token per dot — not one token per channel per dot
- Each channel gets its own weight at that dot, $w_{c,i} = \exp(-\frac{1}{2} \cdot p_i^T \Sigma_c(\ell)^{-1} p_i)$, and it multiplies that channel's value and observed-flag entries **before the projection**. A dot outside channel c 's aperture contributes ≈ 0 for c and still carries the others at full weight
- **Worked example**, a dot 400 km from the anchor at lag 1, apertures at their initial values: currents $\sigma = 259 \text{ km} \rightarrow w = \exp(-\frac{1}{2} \cdot (400/259)^2) \approx 0.30$; sea-surface temperature $\sigma = 500 \text{ km} \rightarrow w \approx 0.73$. The same token feeds temperature at 73 % and the currents at 30 %
- **The gradient reaches S , d and s_c through w from every dot at once** — not through four neighbouring cells — which is what lets the geometry learn (next slide)
- The fixed safety ring (slide 13) is **not gated**: read at full weight, always. A safety net the model can switch off is not one

One token per dot carries every channel; each channel's entries are weighted by its own aperture before the projection

The mechanism that makes a per-channel scale free is the per-location token, which cone v2 introduced in its section 6.4: instead of one token per channel per dot — the E-069 layout, seven hundred and six dot tokens for a handful of channels — one token per dot that carries every channel of the group as a vector of values and a vector of observed flags. Each channel's entries in that vector are multiplied by that channel's own aperture weight at that dot before the token is projected into the model. The weight is a Gaussian of the dot's offset under the channel's covariance: near the centre it is one, far outside the channel's aperture it is essentially zero. So a dot four hundred kilometres from the anchor at lag one is read by the currents at about thirty percent — their initial aperture is two hundred and sixty kilometres — and by sea-surface temperature at about seventy percent, because its initial aperture at that lag is five hundred. Same token, same read, two different weights. No new tokens, no new reads, one elementwise multiply per token. And because the weight is a smooth function of the shared shape, the shared drift and the channel's own offset, the loss reaches all three through every dot at once — a far better gradient than the four-neighbouring-cells one that reading at interpolated positions would give, and the reason the next slide can let the geometry learn without the data loader knowing where it currently is. One exception, stated once: the fixed far ring of cone v2 is never gated. It is the safety net, and a safety net the model can switch off is not one.

A soft aperture over fixed candidate dots in phase 1; the dot table re-baked from the learned ellipses in phase 2

Phase 1 — gate

The candidate dots per region and group are **fixed**: cone v2's log-radial set out to the design reach ([E-071 §4.4](#), 36 per lag) plus the anchor column. The data loader's reads are regular, cacheable, and never depend on the model's weights. Each candidate token is weighted by the per-channel aperture of slide 10, and the ellipses, drift and offsets learn through those weights — from every candidate at once

Phase 2 — harden

Every N steps (2 k to start) the dense dot table per region is re-baked from the current ellipses — the warped sunflower of slide 8, 24 dots per lag — and the loader swaps tables at the epoch boundary. The fixed ring of 24 stays as the safety net; the apertures stay on (they **are** the per-channel scale). Reads remain regular within an epoch

- **Why not read at continuous positions** (the deformable gather of revision 1): it couples the loader's read pattern to the live weights — irregular reads on the sampler's memory-mapped tensor and on the JAX/TPU port of the cone trainer — and its gradient sees only the four cells around each dot. The gate has neither problem, and the per-tensor pyramid of revision 1 is no longer needed
- **Learning rate**: the geometry parameters get their own multiplier (×10 the codec's to start — the deformable-attention practice, [Zhu et al. 2021](#)), recorded in the run's metrics
- Evaluation geometry stays **frozen** (slide 5) whatever phase training is in

A soft aperture over fixed candidate dots in phase 1; the dot table re-baked from the learned ellipses in phase 2

How does a loss reach a geometry, without the data loader having to know where the geometry currently is? In two phases. In the first, the candidate dots are fixed: the log-radial set cone v2 already specified, thirty-six per lag per group out to the design reach, plus the anchor's own column. Because the set is fixed, the sampler reads the same cells for a given region every time — regular, cacheable reads that never depend on what the model has learned so far, which is what the reviewer's comment B was rightly worried about. What the model learns is how much each candidate counts, per channel, through the aperture weights of the previous slide; and because every candidate contributes to those weights, the gradient on the ellipse, the drift and the per-channel offsets is informed by the whole candidate set at once, not by the four cells nearest each dot. In the second phase the geometry is hardened: every couple of thousand steps the dense dot table for each region is regenerated from the current ellipses — the warped sunflower of slide 8 — and the loader swaps to the new table at that boundary, so within an epoch the reads are as regular as before. The far ring stays fixed as the safety net, and the apertures stay on, because they are what carries the per-channel scale. This replaces revision 1's plan of reading at continuous positions by interpolation, which would have made the loader's read pattern depend on the live weights and whose gradient only ever saw four neighbouring cells; it also removes the pre-averaged pyramid that plan needed. One practical detail from the deformable-attention literature: the geometry parameters get their own learning-rate multiplier, ten times the codec's to start, recorded in the run so it can be tuned. And nothing here touches the evaluation geometry, which stays frozen to the fixed hourglass for every arm.

A 1° map per channel group, interpolated to the anchor

- A **1° grid** over the globe ($180 \times 360 = 64,800$ cells); the anchor's parameters are the bilinear blend of the four surrounding map cells — smooth by construction, no seams
- **1° because** the currents that carry the overturning are ~100 km wide (the Florida Current sits in an 80 km strait); a 10° cell would average the Gulf Stream with its recirculation and learn a drift near zero
- **Hierarchical prior:** each map is a coarse 10° map plus a 1° residual, the residual under L2 shrinkage and a Laplacian smoothness penalty — a data-poor cell (ice edge, coast) inherits its neighbourhood's arrow instead of fitting noise. Each 1° cell still sees 16 anchors $\times$ 3,142 pentads $\approx$ 50 k anchor-pentads for eight numbers
- **No Argo ellipse.** The Argo channels are the k nearest profile tokens ([E-071 §3](#), [E-076](#)) — placed where floats were, each carrying its own offset, age and depth. There is no elliptic shape to learn; the search radius stays as specified there. Depth-dependent sourcing (at the RAPID line the upper limb flows north, the Deep Western Boundary Current south) is left to attention over those tokens, which see depth and offset together
- **Optional, phase 2:** a first-harmonic seasonal term on the drift, $d(\tau) = d_0 + d_1 \cos \tau + d_2 \sin \tau$ — the Somali Current reverses with the monsoon

one map per channel group

ocean surface — currents, sea-surface height, sea-surface temperature, mixed layer

atmosphere — drift only at lags 0–1

land — no drift: $v = 0$, axes and angle only

$\approx 1.6 \text{ M}$

geometry parameters, nominal

64,800 cells $\times$ 3 groups $\times$ 8 numbers, about half of it live (ocean maps over ocean, land maps over land), plus ~50 global per-channel scale offsets (slide 9) — cheap, and no compute

A 1° map per channel group, interpolated to the anchor

Where do the eight numbers live? On a one-degree map. A one-degree grid over the globe has about sixty-five thousand cells; an anchor takes the blend of the four cells around it, so the geometry varies smoothly and never jumps at a cell edge. One degree is not a round number picked for comfort: the currents that carry the overturning are about a hundred kilometres wide — the Florida Current runs through an eighty-kilometre strait — and a ten-degree cell would average the Gulf Stream with the water flowing back the other way beside it and learn an arrow of nearly zero. The concern with a fine map is noise in cells that see little data, at the ice edge or along coasts. The answer is a hierarchical prior rather than a coarser map: every map is a coarse ten-degree map plus a one-degree residual, and the residual is shrunk toward zero and smoothed toward its neighbours, so a cell with little evidence inherits the arrow of its region. Even so, each one-degree cell sees about fifty thousand anchor-pentads for its eight numbers. There is one map per channel group, because what moves a signal decides where its source is: the ocean surface group, the atmosphere group with drift only inside its ten-day memory, and the land group, which does not advect and learns only axes and angle. Argo gets no map at all. Its channels arrive as the nearest profile tokens, placed where the floats actually were, each carrying its own offset, age and depth — there is no ellipse there to learn, and the earlier draft's three depth maps were a leftover from the column-only Argo of E-069. The physics that motivated them still matters — at the RAPID line the upper limb flows north and the deep boundary current south — but a token that carries its own depth and offset lets attention weight deep tokens from the north and surface tokens from the south by itself. That becomes a diagnostic rather than a geometry. All told the maps hold about one and a half million numbers, half of them live, plus a few dozen per-channel scale offsets that are global rather than per region, and none of it costs compute.

Bounded by physics, tied to the fixed cone, started isotropic

Speed cap

$|\mathbf{d}| \leq v_{\text{design}} \cdot \Delta t = 5.4 \text{ m/s} \times 5 \text{ d} = 2,333 \text{ km per pentad (E-071's fastest mechanism} \times 1.5)$, via a tanh parameterisation — the learned cone lives inside the v2 envelope

Log-space, with soft floors

Σ_o , Σ_v and s_c are stored as logarithms (slide 8), so Adam's step is a fixed fraction at any size; the waist's eigenvalues are floored at one cell (28 km) and Σ_v 's at the design speed, by a softplus offset in log space rather than a clip

Prior

Coarse $10^\circ + 1^\circ$ residual, L2 shrinkage and Laplacian smoothness on the residual (slide 12) — one weight, recorded in the run's metrics

Mirror tie, by default

The future cone shares the eight numbers (reflected); an untied arm is the ablation, not the design

Stop-gradient on the target side

The loss reaches the geometry only through the **input** reads; the positions of targets are treated as constants

The v2 safety net stays, ungated

In addition to the learned ellipse, each lag keeps one fixed log-radial ring of 24 dots to the antipode, read at full weight — the cone can be directed without asserting that a driver could not have come from elsewhere

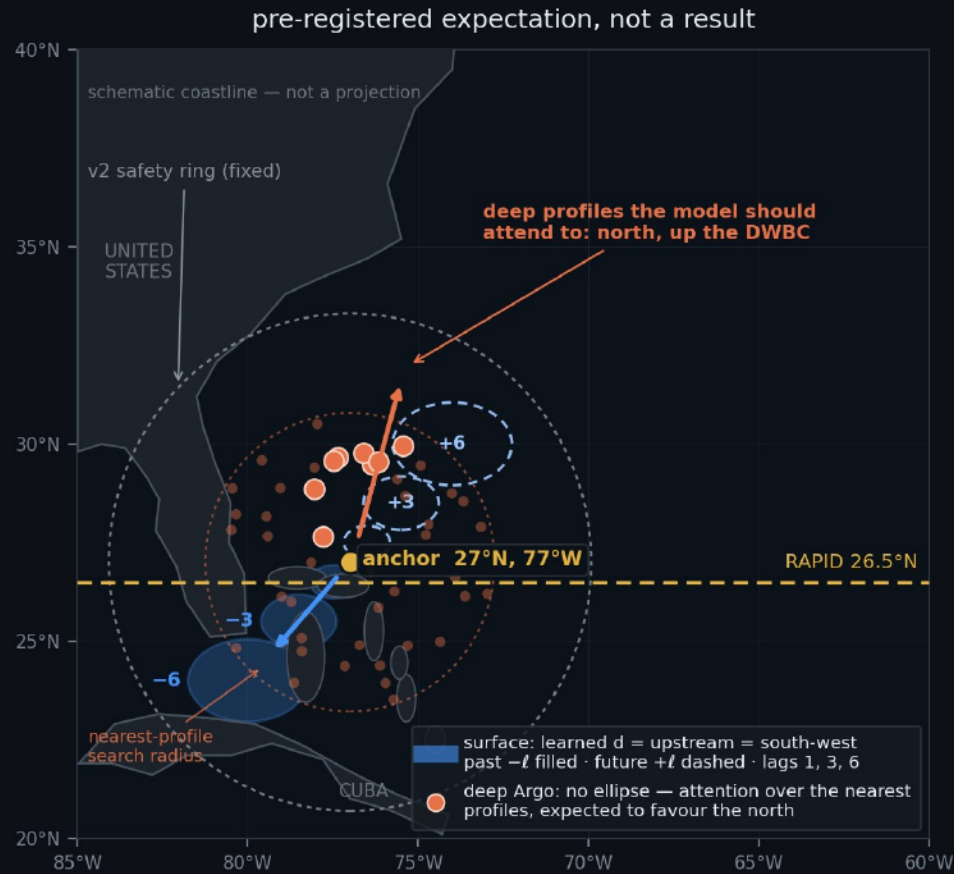
Init

$\mathbf{d} = 0$; S_o = the log of a 3-cell circle with the measured aspect 0.71 ([ml/measure_flow_anisotropy.py](#)); Σ_v from the 0.3 m/s of [ml/cone.py::FAMILIES](#); s_c from the same table per channel — the fixed hourglass exactly, so step 0 of the learned arm is the control

Bounded by physics, tied to the fixed cone, started isotropic

The eight numbers are bounded by physics and tied to the fixed cone. The drift cannot exceed the design speed cone v_2 set — the fastest measured current times one and a half, about two thousand three hundred kilometres per pentad — so the learned cone always lives inside the envelope v_2 argued for. The shapes are stored as logarithms, so the optimiser moves them by fractions rather than by kilometres, and the floors — no waist narrower than one cell, no spread faster than the design speed — are soft offsets in that log space rather than hard clips that would kill the gradient. The one-degree residual is shrunk and smoothed under a single prior weight that the run records. The future cone shares the numbers, reflected; letting it have its own is an ablation. Two guards protect the learning itself. First, the loss reaches the geometry only through the input reads; where a target sits is treated as a constant. Without that, the model would learn to move its questions to wherever they are easiest, and a cone that points at flat water would look excellent. Second, the v_2 safety net stays, and it is never gated: alongside the learned ellipse, every lag keeps one fixed ring of twenty-four dots at log-spaced radii to the far side of the planet, read at full weight. That is the point of v_2 — a cone that is too narrow silently asserts that a driver could not have arrived — and the learned ellipse adds direction and density on top of it rather than replacing it. And the initial values reproduce the fixed hourglass exactly — the measured anisotropy for the waist, the geometry module's speed for the spread, its reach table for each channel's scale — so the learned arm starts as the control and every departure from it is something the data asked for.

One anchor on the western boundary: a learned surface cone, and deep profiles read from the other side



- **Surface group:** the learned d should point south-west (up the Florida Current, toward the Loop Current); the future cone leans north-east along the Gulf Stream
- **Deep Argo:** no ellipse to learn. The profile tokens within the search radius carry their own offset and depth; the model should put its attention on the deep (> 1000 dbar) profiles **north** of the anchor — up the Deep Western Boundary Current — and on the shallow ones to the south
- Neither is programmed: these are the **pre-registered expectations** — a geometry check for the surface, an attention check for the deep
- If the learned surface drift correlates with $-u_{\text{clim}}$ (the climatological surface current, reversed) over the world ocean, the cone learned “upstream” from the data alone

One anchor on the western boundary: a learned surface cone, and deep profiles read from the other side

Here is the picture Chris asked for, drawn as we expect the model to draw it. An anchor on the western boundary just north of the RAPID line. For the surface group, water arrives up the Florida Current from the Caribbean, so the past cone should drift south-west, growing as it goes back, and the future cone should lean north-east along the Gulf Stream. The deep ocean is read differently, because Argo has no ellipse: the anchor sees the nearest profiles within the search radius, each token saying where it was, how old it is and how deep. The Deep Western Boundary Current runs the other way along the continental slope, so if the model has understood the deep ocean, its attention on deep profiles should sit north of the anchor, toward the Grand Banks, while its attention on shallow profiles sits south. Neither is programmed. They are what we expect to see if the mechanism works — a geometry check for the surface, an attention check for the deep — and they are written down now, before any run, so the check cannot be shaped by the result. The world-ocean version of the surface check is a single number: the correlation between the learned surface drift and the climatological surface current reversed, over cells where that current is at least a fifth of a metre per second. A clearly positive value means the cone learned "upstream" from the data alone. And the faint ring around the anchor is the fixed v2 safety net — always there, never learned, so nothing the model does can wall off a direction.

A learnable cone can learn to ask easy questions

The loophole: if target positions could move, the gradient would move them to smooth water — a lower loss that means nothing.

Fix 1 • stop-gradient on target positions

Slide 13. The geometry is trained only by “which inputs help”, never by “which targets are easy”.

Fix 2 • the mirror tie

The future cone cannot be tuned separately from the past cone.

Fix 3 • frozen evaluation geometry

Held-out numbers are always scored on the fixed hourglass, for every arm.

Fix 4 • the aperture never touches the loss weights

In phase 1 the targets are the fixed mirrored candidate set with fixed weights; a model that shrinks its aperture cannot thereby shrink the set of questions it is scored on.

Residual second-order effect: as the input cone moves, the mirrored targets move with it. Monitored, not assumed away — report the target-variance under the learned cone vs the fixed cone at every eval; a drop > 10 % is a flag.

A learnable cone can learn to ask easy questions

One slide on the trap, because it is the kind that produces a beautiful curve and a wrong conclusion. If the model can move where its questions are asked, the gradient will move them to where the answers are easy — smooth open water, away from fronts — and the loss will fall for no reason we want. Three fixes. The target positions are constants as far as the gradient is concerned, so the geometry learns only from which inputs help. The future cone is tied to the past cone, so it cannot be tuned on its own. And every held-out number is scored on the frozen hourglass, the same for every arm. A fourth guard follows from the gate design: the learned aperture weights the inputs only, never the loss — the targets in phase one are the fixed mirrored candidate set with fixed weights, so a model that narrows its aperture gains nothing on the questions it is scored on. There is a residual, second-order version of the trap: as the input cone moves, the mirrored targets move with it. We do not assume that away. At every evaluation the run reports the variance of its targets under the learned cone against the fixed cone, and a drop of more than ten percent is a flag that the geometry is drifting toward easy water rather than toward the source.

Tokens, gathers, parameters — the same regime as v2

	E-069 (measured)	this design (budget)
waist	42 patch tokens (14 ch × 3×3)	13 per-location tokens carrying all surface channels
past cone	706 dot tokens	phase 1: 6 lags × (1 + 36 candidates) × 4 groups ≈ 888 · phase 2: 6 × (1 + 24 ellipse + 24 ring) × 4 ≈ 1,176
Argo	192 (32 × 6, column only)	26 profile tokens (E-071 §3)
targets	42 + 84	256 dot queries per example, drawn from the future cone and the hidden waist / dots (as n_dot_queries today)
geometry parameters	0	≈ 1.6 M nominal, ~half live (slide 12); no compute
extra per-tensor artefact	—	none (revision 1's pyramid is gone); the per-region dot tables re-baked at epoch boundaries are small

- Perceiver cost is linear in tokens: ~1.2× (phase 1) to ~1.6× (phase 2) E-069's encoder time, inside v2's own 2.1× estimate; the aperture weights are one elementwise multiply per token
- Training cost per seed: E-069 seeds ran under one 4090-hour on the North Atlantic; budget 2–3 4090-hours per seed on the global tensor

Tokens, gathers, parameters — the same regime as v2

What it costs, in the same units as the earlier decks. The waist is thirteen per-location tokens each carrying all surface channels, which is fewer than the forty-two per-channel patch tokens E-069 used. The past cone is, in the gate phase, six lags of one anchor column and thirty-six fixed candidates for four channel groups — about nine hundred tokens — and after hardening six lags of the column, twenty-four ellipse dots and twenty-four ring dots — about twelve hundred. Argo comes as the twenty-six profile tokens cone v2 already specified. Targets are drawn as two hundred and fifty-six decoder queries per example, as now. The geometry adds about one and a half million parameters, half of them live and none of them costing compute, and no new per-tensor artefact — the pyramid of revision 1 is gone, and the re-baked dot tables are small. Because the encoder is a Perceiver, its cost is linear in the token count, so this is between one point two and one and a half times E-069 per step — inside the two-times cone v2 already budgeted for itself. Per seed, E-069 ran under one GPU-hour on the North Atlantic; on the global tensor, budget two to three. Nine runs for the experiment on the next slide is on the order of twenty-five GPU-hours.

Three arms, three seeds, four read-outs, one verdict rule

A0 • fixed

The hourglass with the v2 geometry (isotropic ellipses, ring, mirror), no learning — the control, and the eval geometry for everyone

A1 • learned

The eight numbers per region plus the per-channel scales, gate-then-harden (slide 11), initialised at A0

A2 • learned, primed

As A1, but **d** initialised at $-u_{\text{clim}} \cdot \Delta t$ (the climatological surface current, reversed) for the ocean surface group — tests whether the gradient can find upstream on its own (A1) or only keep it (A2)

Held fixed: same tensor (family 7.2, global), same 7 M codec, 20 k steps, frozen protocol (train $\leq$ 2020; 2008–09, 2016–17, 2021–24 held out), 3 seeds each.

R1 T1 future-cone loss per lead vs bar • **R2** the T1 – T2 gap (what history buys) • **R3** drift-vs-upstream correlation, world ocean, $|u_{\text{clim}}| > 0.2$ m/s

R4 the western-boundary depth check — surface drift south-west (geometry); attention mass on deep (> 1000 dbar) profile tokens north of the anchor, on shallow ones south (attention diagnostic)

R5 target-variance flag (slide 15)

Verdict: A1 adopted if it beats A0 on R1 at every lead ≤ 3 , paired at all three seeds, with R5 clean. $A2 > A1$ with $A1 \approx A0$ means the gradient cannot find upstream — then geometry is set from climatology, not learned. Neither $\rightarrow$ keep A0 and move to a dynamic (per-example) cone. **First ablation after the arms:** A1 without the far ring — does the ring pay for its tokens?

Three arms, three seeds, four read-outs, one verdict rule

The experiment, written before anything runs. Three arms. The fixed hourglass with cone v2's geometry is the control and also the evaluation yardstick for every arm. The learned arm starts from the control. The primed arm starts its surface drift at the climatological current reversed, which separates two questions: can the gradient find upstream on its own, or can it only keep upstream if handed it? Everything else is held fixed — the global family 7.2 tensor, the seven-million codec, twenty thousand steps, the frozen protocol with its held-out years, three seeds per arm because the read-outs are probe-scored. Five read-outs: the forecast loss per lead against its bar; the gap between the full task and the snapshot task, which is what history buys; the correlation of the learned drift with upstream over the world ocean; the depth check at the western boundary — the surface drift read from the map, the deep direction read from where the model puts its attention among the profile tokens; and the target-variance flag. The verdict rule is written now. The learned cone is adopted if it beats the control on the forecast at every lead up to three, paired at all three seeds, with the flag clean. If only the primed arm wins, the gradient cannot find upstream and geometry should be set from climatology rather than learned. If neither wins, the fixed hourglass stays and the next design is a dynamic cone — one whose drift is predicted per example from the waist — not a bigger static one. And the first ablation after the three arms is the one the reviewer and I disagree on: the same learned arm without the far ring, which measures whether the ring pays for its tokens rather than arguing it.

Five steps, each with its test

in ml/cone.py · ml/cone_sampler.py · ml/cone_codec.py · ml/train_cone.py

- 1 **Waist + future cone in the geometry (cone.py)** waist_dots, future_dots = reflected inner_dots; coverage_report extended with the mirror identity (future set = -past set) — pure numpy, CPU test
- 2 **The task mix (cone_codec.py::default_plan → plan_v3)** T1-T4 with shares and the five dropout patterns; test: every target's channel is visible somewhere in the input, for 10 k drawn examples, zero violations
- 3 **The aperture gate (cone_codec.py)** per-channel weights $w_{\{c,i\}}$ on the per-location candidate tokens, ring ungated; test: with every $s_c \rightarrow +\infty$ the forward pass equals the ungated model bit-for-bit, and the finite-difference gradient of the loss w.r.t. S_0, Σ_v, d, s_c matches autograd
- 4 **The parameter maps and the harden step (cone_codec.py::ConeGeometry, cone_sampler.py)** 1° maps per group as coarse 10° + residual, matrix-log storage, the log-space floors, the prior, mirror, stop-gradient, init = A0; the sampler's dot table becomes a per-region lookup swapped at epoch boundaries; test: the table re-baked at init equals step 1's fixed dots to < 0.1 cell, and a swap mid-run changes no read of the current epoch
- 5 **Read-outs in the trainer (train_cone.py)** per-task bars, R3 drift maps exported as data/cone_drift_<group>.json for the app's Cones tab, R4 attention-by-depth-and-bearing at the western-boundary anchors, R5 flag; then the three arms as recipes f7l2-hourglass-{fixed,learned,primed}

Each step lands on main with its test before the next starts; step 4 is the only one that touches the data path, and only at epoch boundaries.

Five steps, each with its test

The build order, for whoever implements it. Five steps, each small enough to land with its own test before the next begins. First the geometry: the waist and the future cone as functions in the cone module, with the coverage report extended to assert the mirror identity — pure numpy, testable on a laptop. Second the task mix, as a new masking plan next to the existing one, with a test that draws ten thousand examples and finds zero targets whose channel is invisible. Third the aperture gate in the codec: per-channel weights on the candidate tokens, with two tests — an infinite aperture must reproduce the ungated model exactly, and the gradient of the loss with respect to every geometry number must match finite differences. Fourth the parameter maps and the harden step — coarse plus residual, the matrix-log storage, the log-space floors, the prior, the mirror, the stop-gradient, the control initialisation, and the sampler's dot table as a per-region lookup that is only ever swapped at an epoch boundary — with a test that the table re-baked at initialisation equals the fixed dots and that a swap never changes a read inside the current epoch. Fifth the read-outs in the trainer, including exporting the learned drift maps to the app so the Cones tab can draw them, and the attention-by-depth diagnostic at the western-boundary anchors. Then the three arms are three recipes on the global tensor, and E-080 is dispatched under the usual rules.

Two settled, six still open — all reversible

Settled 15 Sep (Chris): 1° parameter maps with the coarse-plus-residual prior; **no Argo ellipses** — the nearest-profile tokens stay as E-071 §3 specifies them.

Future horizon

+6 pentads to mirror the past; or shorter (+1...+2 as E-069) with denser targets?

Task shares

0.35 / 0.15 / 0.25 / 0.25 — any of them a lever you want set differently?

Prior strength on the 1° residual

One weight, to be set by held-out loss on the training years — or fixed by hand for the first arm?

The v2 safety ring

Kept at every lag (24 tokens per lag per group); dropping it halves the past-cone tokens but restores v2's "too narrow" risk

Seasonal drift term

Deferred to phase 2; or in from the start (monsoon regions)?

Harden interval and geometry LR

N = 2 k steps and the geometry learning-rate multiplier **×10** — first guesses, both recorded per run

Two settled, six still open — all reversible

Two choices are settled: the maps are at one degree with the coarse-plus-residual prior, and Argo keeps its nearest-profile tokens with no ellipse. Six remain, each one line to change. The future cone goes to six pentads to mirror the past, where E-069 only asked for one and two. The task shares are a first guess. The strength of the prior on the fine map is one number, and the honest way to set it is by held-out loss on the training years rather than by hand — but the first arm may simply fix it. The v2 safety ring is kept at every lag, which is the safe and more expensive choice. The seasonal term on the drift waits for phase two. And the harden interval and the geometry learning-rate multiplier are first guesses that every run records. Say the word on any of these and the spec changes; none of them affects the build order.

Sources

the record this deck stands on

- **E-069 · Two stencils, one cone** — [ml/plans/E069_cone_codec.md](#); results in [ml/EXPERIMENTS.md#e-069](#) (five seeds, H1 refuted; what history buys)
- **E-069b masking-plan fix** — [ml/cone_codec.py::default_plan](#) (chan_drop_scope, anchor_hidden_only), [ml/train_cone.py](#)
- **E-071 · Cone v2** — [ml/plans/E071_cone_v2.md](#) (\$3 profile tokens, \$4 design speeds and log-radial dots, \$4.5 the split in time, \$6.4 per-location tokens)
- **E-076 · Family 8 nearest observations** — [ml/plans/E076_family8_nearest_observations.md](#) (the (value, Δx , Δy , Δt , n_R) token)
- **The geometry as built** — [ml/cone.py](#) (FAMILIES, reach_km, slots, inner_dots, outer_spiral, coverage_report)
- **E-026 · the sunflower stencil** — [ml/temporal.py::spiral_offsets](#), the pattern the warped sunflower reuses

and on

- **The geometry learning-rate multiplier and soft sampling weights:** [Zhu et al. 2021, Deformable DETR](#)
- **The step-size argument for log-space parameters:** [Kingma & Ba 2015, Adam](#) — the update is $\approx lr$ per step in the parameter's own units
- **The matrix logarithm of a symmetric 2×2 covariance:** $S = m \cdot I + k \cdot [\cos 2\theta, \sin 2\theta; \sin 2\theta, -\cos 2\theta]$ with $m = \ln \sigma_1 + \ln \sigma_2$, $k = \ln \sigma_1 - \ln \sigma_2$ — standard, no reference needed
- **Currents:** E-071 \$4.1's table (Somali 3.6 m/s, Gulf Stream ~ 2.5 , DWBC 0.02–0.1 m/s); RAPID 26.5°N array for the limb directions

Sources

Everything the deck leans on. The E-069 plan and its five-seed record; the E-069b masking fix in the codec; cone v2 for the design speeds, the profile tokens and the per-location tokens; family 8 for the token that carries its own offset; the geometry module for every constant. The learning mechanism borrows two things from the deformable-attention literature — soft sampling weights and a separate learning rate for the geometry — and the scale-free storage rests on how the Adam optimiser sizes its steps; the matrix-logarithm form of an ellipse is textbook linear algebra. The current speeds and the directions of the two AMOC limbs are the ones cone v2 already tabulated and the RAPID array's own description of the section.